Remymy

An agentic conversational meal-ranking system

Pattern Recognition Systems

Final project

Group 44

Le Chi Thanh / Vuong Quang Viet Tung / Tran Dinh Gia Khanh



The recommendation problem

Preference-based ranking

“Quick, filling, and not too heavy.”

The interface captures intent. Rated history guides the ordering of candidate meals.

Model requirements

Many catalog items have no Food.com ratings.

User tastes change across time.

Diet and budget constrain the eligible pool.

PRS aspects: supervised learning, Transformer deep learning, and hybrid retrieval with deterministic rules.

System architecture



Conversation and recipe presentation

Catalog, history, preferences and the served reranker

Persistent records and recommendation cache

Optional recommender deployment



Separate Docker profile for the earlier service split

Serving a recommendation

`GET /api/recommend/{username}?top_k=10`

01	**Load history**	Read the user's newest 50 ratings and preferences, and cluster them
02	**Retrieve and score**	Filter eligible items, retrieve candidates, then run the reranker (compared to every cluster).
03	**Return results**	Return a buying profile and up to top_k IDs, titles and scores.
04	**Cache results**	Cache for one day. Clear on preference changes and every 6th rating.

A user with no ratings receives an empty recommendation list.

Data preparation for unseen recipes

19,962

items in the reported serving catalog

16,878

had no Food.com ratings

Content features replace item IDs

GLiNER extracts product, adjective and verb terms.

Lemmatize and singularize into up to 40 tokens.

Use a 2,562-token vocabulary plus field embeddings.

Transform price and 7 nutrients with log1p and training statistics.

Reported catalog includes 10,658 FoodData Central items and 6,220 Open Recipes.

Retrieval narrows the scoring problem

19,962 items → 100 candidates

Retrieve

HNSW cosine distance based on user's history

M = 16, ef = 200

Rerank

Candidate queries the profile and rating states

MLP returns P(like)

Diversify

MMR with λ = 0.5

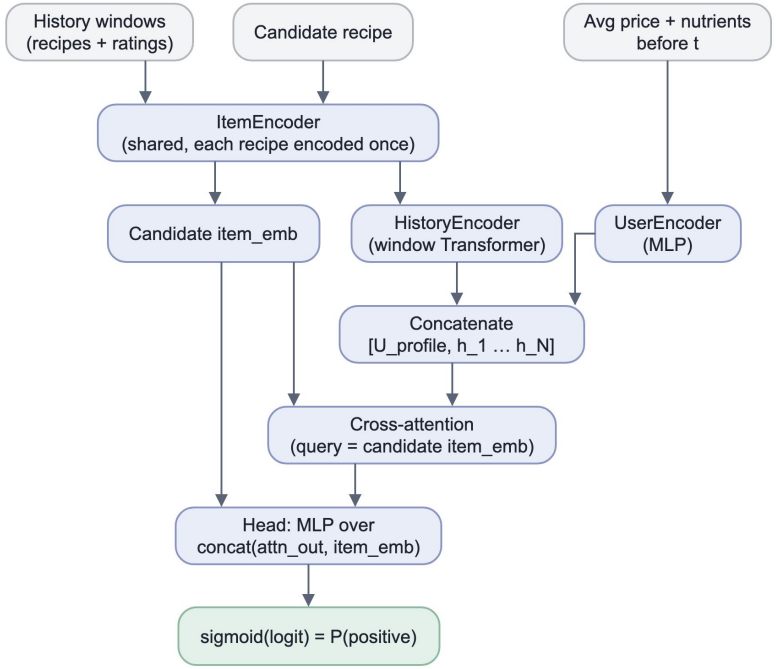
Keep up to 50 items

Drop near-duplicates

Cosine similarity cutoff: 0.97

Diet and budget restrict retrieval. Narrower eligible pools can use an exact scan.

General model architecture



Shared ItemEncoder

Encodes both candidate recipes and the user's rated history.

Candidate-specific attention

The candidate queries history and the user profile before an MLP returns P(like).

Item and user profile encoders

ItemEncoder

Self-attention over ingredient terms

Mean pooling, no positional encoding

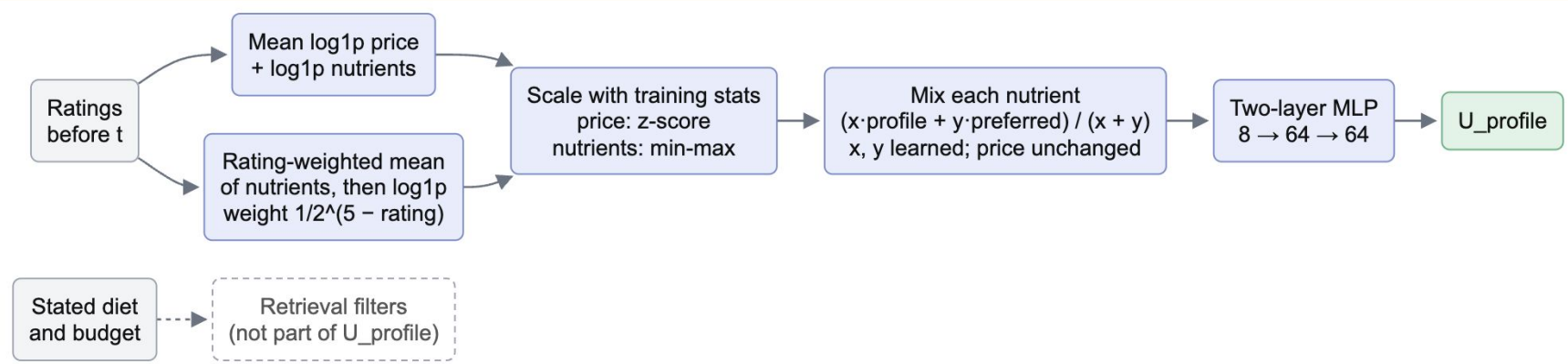
64-d vector from text and numbers

UserEncoder

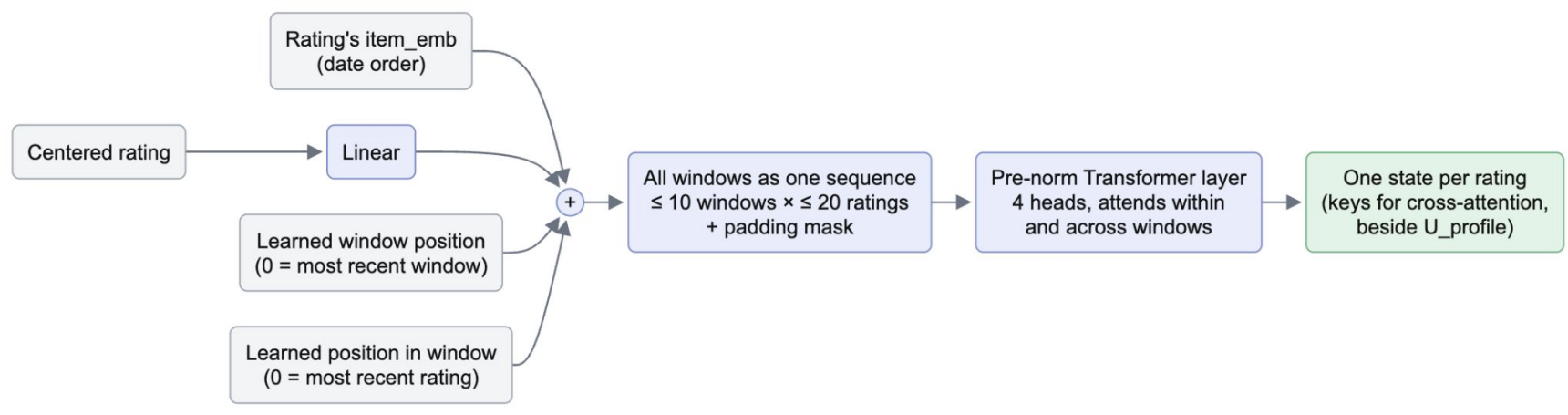
Summarizes long-run price and nutrition

Higher ratings receive more weight

Learned nutrient mixing feeds an MLP



Windowed history and candidate attention



History windows

183-day gaps start new windows.

Use 10 windows × 20 ratings.

Center ratings on the user’s mean.

Candidate-specific scoring

One state per rating.

4-head cross-attention reads history plus the learned profile token.

Training with temporal holdouts

Ratings and sampled negatives

807,076 ratings from 18,888 users with at least 6 ratings

3–5 stars: positive. 0–2 stars: negative or unrated (negative sampling).

For each positive rating, we pair with one unrated recipe by user.

Unrated popular items have more chance to be sampled

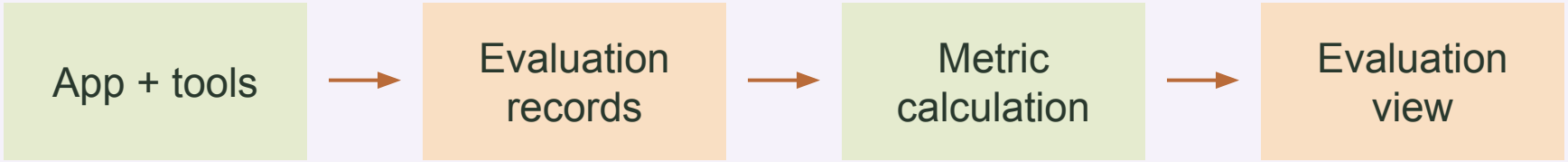
Recorded test results

17,059 held-out positives, each ranked against 99 unrated recipes

Metric	Value
Log loss	0.613
ROC-AUC, all rows	0.718
HR@10	0.319
Mean rank (of 100)	28.5

The sampled ranking task is directional. Food.com recipes and FDC foods use different nutrient and price bases, so P(like) serves as a ranking signal.

System evaluation needs labels and logs



Explicit ranking labels

Recipe relevance grades: 0 to 3
nDCG@6: higher relevance earlier
Precision@6: share with grade ≥ 2
Pairwise: order of unequal grades

Observed user behavior (TBU)

Log impressions and save/open actions.
HR@6: a positive in the top 6
MRR@6: first-positive rank
Track p50/p95 latency and failures.

These remain planned system readouts until relevance labels and mature feedback logs exist.

Jev evaluation: input and output

Input (excerpt)


```
{
  "model": "jev-latest",
  "state": {
    "request": {
      "preferences": {
        "diet": "vegetarian",
        "maxMinutes": 30,
        "exclude": ["peanuts"],
        "goal": "balanced",
        "cuisine": "any"
      },
      "query": "Suggest a vegetarian dinner"
    },
    "candidate_recipe": {
      "recipe_id": "foodcom:12345",
      "title": "Chickpea Spinach Curry",
      "ingredients": ["chickpeas", "spinach", "coconut milk", "curry
powder"],
      "instructions": ["Heat oil...", "Add spinach..."]
    }
  },
  "questions": {
    "fit_grade": {
      "type": "literal",
      "choices": ["0", "1", "2", "3"]
    }
  }
}
```


Output (actual)


```
{
  "model": "jev-1.14.0",
  "answers": {
    "fit_grade": {
      "type": "literal",
      "literal": "3"
    }
  },
  "usage": {
    "input_tokens": 1420,
    "output_tokens": 15
  }
}
```

Demo



Thank you

